hey claude this is my project folder zip file where it has all the main code files. Your task is to extract the all the function calls which are called in a flow to complete the task or project tell me in two phases one is training and other is GUI and complete project end to end dont forget to arrange function calls in an order that execute Uploaded files: - sonar_ship_detection - Copy (2).zip (201.6 MB)

I've read all the training and GUI entry files. Now I'm checking the animated widget classes (timers and paint calls) so the GUI call order is accurate.

I read all 15 source files in the zip. The training flow has one gap: no script that generates train.npz, val.npz or v2_train.npz is included. I mark that spot below.

The project is binary ship detection (ship / no ship) on 5-second sonar clips. Each clip is turned into a mel spectrogram and classified by a ResNet-18 (or EfficientNet-B0) with one output logit.

PHASE 1: TRAINING (in execution order)

Step 0: Shared setup (imported by everything)

```
sonar_config.py
  ├─ constants: SAMPLE_RATE=8000, DURATION=5s, N_SAMPLES=40000, N_FFT=1024,
  │             HOP=313, N_MELS=128, BATCH=64, EPOCHS=20, LR=1e-3
  ├─ get_device()    → cuda / mps / cpu
  └─ make_dirs()     → creates data/, checkpoints/, results/
```

Step 1: Synthetic data generation (simulator.py, simulator_v2.py)

v1 simulator, called per clip via sim.make_sample(rng, label):

```
make_sample(rng, label, snr_range=(-15, 5))
  ├─ ambient_noise(rng)                    ← class 0 base, also the background for class 1
  │    ├─ colored_noise(rng, slope)  → _normalize()
  │    ├─ bandpass()                 → distant-shipping rumble
  │    ├─ slow sea-state swell
  │    └─ hard negatives (random pick):
  │         ├─ _whale_calls(rng)
  │         ├─ _clicks(rng)
  │         └─ _interference_tone(rng)
  ├─ if label == 1:
```

```
│   ├─ bandpass(amb, 20, 2500)          → measure ambient level
│   ├─ ship_signature(rng)              → harmonics + DEMON AM + cavitation (uses bandpass, _normalize)
│   └─ mix at random SNR
└─ returns (waveform, snr_db)

to_spectrogram(wave)
├─ librosa.feature.melspectrogram()
├─ librosa.power_to_db()
└─ per-sample standardisation → (128×128) float32
```

v2 simulator (harder data, used for checkpoints_v2), s2.make_sample(rng, label, snr_range=(-20, 5)):

```
├─ sim.ambient_noise(rng)
├─ (50%) impulsive(rng)              → broadband burst noise
├─ if label == 1: ship_v2(rng)       → 3 ship types (broadband / high-freq tonal / original ship_signature)
└─ random_shaping(rng, wave)         → random EQ tilt, bumps, low-cut
```

Missing script (inferred): something loops make_sample() → to_spectrogram() and saves data/train.npz, val.npz, test.npz (keys X, y), using the 70/15/15 split in config. For v2 it saves data/v2_train.npz. If you have that file, send it and I'll add it to the flow.

Step 2: Real-data preparation (ShipsEar dataset)

real_prepare.py → main()

```
├─ ROOT.rglob("*.wav")
├─ for each file:
│   ├─ librosa.load(sr=8000, mono=True)
│   ├─ pad / crop to 40000 samples, skip silent clips
│   ├─ sim.to_spectrogram(wave)  → stored as float16
│   └─ label: class 4 (background) → 0, classes 0–3 (ships) → 1
└─ np.savez → data/real_all.npz  (X, y, cls, grp, seg)
```

real_split.py (runs on import, no main()):

```
├─ np.load(real_all.npz)
├─ per recording group: sort by segment, first 70% → dev, last 30% → holdout
└─ np.savez → data/real_dev.npz, data/real_holdout.npz
```

Step 3: Base training (train.py → main())

Run as: python train.py --arch resnet18 --ckpt-dir checkpoints_v2 --train-file v2_train ...

```
main()
 ├─ argparse (arch, epochs, batch, lr, patience, ckpt-dir, train/val file)
 ├─ config.make_dirs()
 ├─ torch.manual_seed / np.random.seed
 ├─ config.get_device()
 ├─ load_split(train_file) , load_split(val_file)      → tensors on device
 ├─ sonar_model.build_model(arch, pretrained)
 │     └─ _build()   → ResNet-18 / EfficientNet-B0, conv1 changed to 1 channel, head → 1
logit
 ├─ sonar_model.count_params(model)
 ├─ BCEWithLogitsLoss, AdamW, OneCycleLR, GradScaler
 ├─ for epoch in 1..EPOCHS:
 │     ├─ model.train()
 │     ├─ for each batch:
 │     │     ├─ augment(x)         → random circular time-shift + time-mask
 │     │     ├─ model(x) → loss → backward → scaler.step → sched.step
 │     ├─ evaluate(model, Xva, yva, loss_fn)   → val loss, val acc
 │     ├─ if val loss improved → torch.save(checkpoints*/{arch}_best.pt)
 │     └─ else patience counter → early stop
 ├─ json.dump → results/history_{arch}.json
 └─ print best epoch
```

Output: checkpoints/resnet18_best.pt (and efficientnet_b0_best.pt). The checkpoints_v1/ and checkpoints_v2/ folders come from runs with different data and settings.

Step 4: Mixed synthetic and real fine-tuning (train_mixed.py → main())

This is an optional stage that adapts the v2 model to real data.

```
main()
 ├─ argparse (epochs, lr, syn, cap, noise-rep, groups, --final)
 ├─ np.load(v2_train.npz), np.load(real_all.npz)
 │
 ├─ if --final:                          → final model on ALL real data
 │     ├─ load_v2(device)       → loads checkpoints_v2/resnet18_best.pt via build_model()
 │     ├─ finetune(m, sX, sy, X, y, ...)
 │     └─ torch.save → checkpoints_v3/resnet18_best.pt
 │
```

```
└─ else: leave-one-ship-group-out cross-validation, for each held-out group g:
    ├─ build train/test indices (ships of other groups + separated noise blocks)
    ├─ load_v2() → base;   get_logits(base, X[te])       → baseline scores
    ├─ load_v2() → mixed
    ├─ finetune(mixed, ...)                              → adapted model
    │    └─ per epoch: sample synthetic + real ships + repeated real noise
    │         per batch: spec_aug(augment(xb)) → forward → loss → backward → step
    │              (augment is imported from train.py)
    ├─ get_logits(mixed, X[te])
    ├─ score(z) → roc_auc_score + detection rate @ 10% false alarms
    └─ json.dump → results/logo_results.json
```

PHASE 2: GUI / INFERENCE (python sonar_app.py)

Step 1: Startup (module import time)

```
sonar_app.py imports
 ├─ sonar_config, simulator (sim), simulator_v2 (s2)
 ├─ sonar_model.build_model
 ├─ infer.split_windows
 ├─ gui_theme (th)
 └─ gui_extras → gui_widgets
     PingHeader(WaveHeader), AlertRadar(SonarRadar), HydrophoneStrip, AlertBanner,
TitlePanel
 LUT = magma colormap (256 colours)   ← used to colour spectrograms
```

Step 2: main()

```
main()
 ├─ QApplication(sys.argv)
 ├─ app.setStyleSheet(th.STYLE)
 ├─ win = Main()        ← builds the UI (below)
 ├─ win.show()
 └─ app.exec()          ← Qt event loop
```

Step 3: Main.__init__() builds the window

```
Main.__init__()
 ├─ PingHeader("PASSIVE SONAR", ...)     → WaveHeader.__init__ → QTimer(16 ms) →
_tick() → paintEvent()
 ├─ AlertBanner()                         → QTimer → _tick()
 ├─ AlertRadar()                          → SonarRadar.__init__ → QTimer → _tick()
```

```
├─ HydrophoneStrip()                → QTimer(30 ms) → _tick()
├─ buttons: Load audio / Demo: ship / Demo: ocean only
├─ QComboBox: add each checkpoint that exists:
│     v2 (checkpoints_v2), 37k simulator (checkpoints), 7k (checkpoints_v1), EfficientNet-B0
├─ sliders: threshold (5–95), demo SNR (−20..5)
├─ TitlePanel(...)
├─ SpecView()      ← spectrogram canvas
├─ QProgressBar, QTableWidget (Window | P(ship) | Verdict)
├─ signal wiring:
│    btn_load  → pick_file
│    btn_ship  → run_demo(1)
│    btn_none  → run_demo(0)
│    thr slider → on_threshold
│    combo     → update_thr_label
│    table selection → on_select
├─ update_thr_label()
└─ if no checkpoints found → QMessageBox warning, disable buttons
```

Step 4: User action → analysis pipeline

A. Button click

```
Load audio:   pick_file()  → QFileDialog → start({"kind":"file","path":...})
Demo buttons: run_demo(label) → start({"kind":"demo","label":0|1,"snr":...,"seed":...})
```

B. start(job)

```
start(job)
├─ guard: return if a worker is already running
├─ read (arch, ckpt_dir) from combo; is_v2 = ckpt dir ends with "checkpoints_v2"
├─ set_busy(True)            → disable buttons
├─ set_visual("scanning")     → radar.set_state / banner.set_state / strip.set_ship(False)
├─ progress bar reset + _style_bar(PRIMARY)
├─ Worker(job)               ← QThread
└─ worker.start()  →  Worker.run()
```

C. Worker.run() (background thread, the core inference)

```
Worker.run()
├─ get audio:
│    ├─ demo:  s2.make_sample() (if v2) or sim.make_sample()  → waveform
│    └─ file:  librosa.load(path, sr=8000, mono=True)
├─ check length ≥ 1 s
├─ config.get_device()
```

```
├─ get_model(arch, ckpt_dir, device)      ← cached in _MODELS
│    ├─ torch.load(checkpoint)
│    ├─ build_model(arch, pretrained=False)
│    └─ load_state_dict() → eval()
├─ for (t0, chunk) in infer.split_windows(wave):   ← 5 s windows, pads last piece if ≥ 2.5 s
│    ├─ sim.to_spectrogram(chunk)
│    ├─ model(spec) → logit z
│    └─ prob = sigmoid(z); store start, prob, spec, logit
├─ emit done(dict(starts, probs, specs, logits))
└─ on error: emit failed(msg)
```

D. Result handling (main thread)

```
on_done(res)       → QTimer.singleShot(wait to finish ~1.6 s scan animation)
  └─ show_result(res)
       ├─ self.sel = argmax(probs)   → window with highest score
       ├─ set_busy(False)
       └─ apply_results()
            ├─ if v2 model: cutoff = (9.23 for real files, 0 for demo) + slider shift,
            │              probs = sigmoid(logit − cutoff), threshold = 0.5
            │  else:       threshold = slider / 100
            ├─ fill table rows (window time, score, SHIP / no ship, coloured)
            ├─ table.selectRow(sel)
            ├─ peak = max(probs); ship = peak > thr
            ├─ set_visual("ship" | "clear", peak)  → radar, banner, strip update
            ├─ progress bar = peak %, colour red/green (_style_bar)
            └─ spec_view.set_spec(specs[sel])
                 └─ normalise → magma LUT → QImage → update() → SpecView.paintEvent()

on_failed(msg) → set_busy(False), set_visual("idle"), QMessageBox.critical
```

E. Interactions after results

```
Threshold slider  → on_threshold(v) → update_thr_label() → apply_results()   (re-score
without re-running the model)
Table row click   → on_select()     → spec_view.set_spec(specs[row])
Model dropdown    → update_thr_label()
Window close      → closeEvent() → worker.wait()
```

Optional: command-line inference (infer.py)

```
main() → config.make_dirs() → (demo: sim.make_sample + wavfile.write | file: path)
 → librosa.load → load_model(arch) → for each window from split_windows():
predict_window() → print verdicts
```

infer.py is not required by the GUI except for split_windows.

End-to-end summary

```
[TRAINING]
 simulator / simulator_v2 → (missing dataset-build script) → data/{train,val,v2_train}.npz
 real_prepare.py → real_all.npz → real_split.py → real_dev / real_holdout.npz
 train.py        → checkpoints*/resnet18_best.pt
 train_mixed.py  → fine-tune on synthetic + real → checkpoints_v3 (--final)

[GUI]
 sonar_app.main → Main.__init__ → start → Worker.run
   → split_windows → to_spectrogram → model → sigmoid
 → show_result → apply_results → table / radar / banner / progress bar / spectrogram
```

Notes:

- The GUI never trains. It only loads .pt files from checkpoints_v2/, checkpoints/ and checkpoints_v1/. checkpoints_v3 isn't in the dropdown.
- exec.txt shows the intended run order: create the venv, install packages, check the checkpoint, then run python sonar_app.py.
- gui_widgets.py and gui_extras.py are visual only, with timers driving the animations and no ML calls.

I can turn this into a flowchart diagram or a Word/PDF document if you want.

> ok ok now just list out the function calls that executed while i running train.py

Function calls executed when you run python train.py

Command: python train.py --arch resnet18 --train-file train --val-file val --ckpt-dir checkpoints

0. Import time (before main)

```
os.environ.setdefault("PYTORCH_ENABLE_MPS_FALLBACK", "1")
import sonar_config as config      → Path(__file__).resolve().parent, defines get_device() and make_dirs()
from sonar_model import build_model, count_params
```

1. if __name__ == "__main__" → main()

```
1.  argparse.ArgumentParser() → add_argument() ×8 → parse_args()
2.  config.make_dirs()                    → creates data/, checkpoints/, results/
3.  torch.manual_seed(config.SEED)
4.  np.random.seed(config.SEED)
5.  config.get_device()                   → cuda / mps / cpu
6.  load_split("train", device)           → np.load(data/train.npz) → tensors on device
7.  load_split("val", device)             → np.load(data/val.npz)  → tensors on device
8.  build_model("resnet18", pretrained=True)

    └─ _build("resnet18", True)           → models.resnet18(), new 1-channel conv1, new fc
→ 1 logit
    └─ (if pretrained download fails: prints warning, calls _build("resnet18", False))

9.  model.to(device)
10. count_params(model)                   → printed in the "Arch: ... params" line
11. nn.BCEWithLogitsLoss()
12. torch.optim.AdamW(...)
13. math.ceil(len(Xtr) / batch)           → steps per epoch
14. torch.optim.lr_scheduler.OneCycleLR(...)
15. torch.amp.GradScaler("cuda", enabled=use_amp)
16. ckpt_dir.mkdir(exist_ok=True)
```

2. Epoch loop (repeats up to 20 times, or until early stop)

```
for epoch in 1..EPOCHS:
 ├─ model.train()
 ├─ torch.randperm(len(Xtr))
 │
 ├─ for each batch (64 samples):
 │    ├─ augment(Xtr[idx].unsqueeze(1).float())
 │    │    └─ torch.randint, torch.arange, x.gather()    → circular time shift
 │    │    └─ torch.randint, torch.rand, masked_fill()   → random time mask
 │    ├─ opt.zero_grad()
 │    ├─ torch.autocast(...)
 │    │    ├─ model(xb)                    → forward pass
 │    │    └─ loss_fn(out.float(), yb)
 │    ├─ scaler.scale(loss).backward()              → backprop
 │    ├─ scaler.step(opt)
 │    ├─ scaler.update()
 │    └─ sched.step()
```

```
│
├─ evaluate(model, Xva, yva, loss_fn)
│     ├─ model.eval()
│     └─ per batch of 256: model(xb) → loss_fn() → accuracy count
│
├─ history.append(...) and print the epoch row
│
└─ if va_loss < best_val:
│       torch.save({... state_dict ...}, checkpoints/resnet18_best.pt)
│  else:
│       bad += 1 → if bad ≥ patience (5): print "Early stopping" → break
```

3. After training

```
json.dump(history, results/history_resnet18.json)
min(history, key=val_loss)          → best epoch
print(best epoch, checkpoint path)
```

Compact order

```
config.make_dirs → manual_seed → get_device → load_split ×2 → build_model(_build) →
count_params
→ BCEWithLogitsLoss → AdamW → OneCycleLR → GradScaler
→ [ model.train → randperm → (augment → forward → loss → backward → step →
sched.step) × batches
   → evaluate → torch.save (if best) ] × epochs
→ json.dump → print best result
```

train.py does not call anything from simulator.py, infer.py or any GUI file. It only needs data/train.npz and data/val.npz to exist already.

ok extract those code files and explain each lines why we written those lines what they will do

I'm taking "those code files" to mean the three files train.py actually runs: sonar_config.py, sonar_model.py and train.py. They are explained in the order Python loads them.

FILE 1: sonar_config.py (settings and helpers)

This file holds every setting in one place, so you change a value here instead of hunting through the code.

```
from pathlib import Path
import torch
```

- **Line 1:** Path handles file paths in a way that works on both Windows and Mac.
- **Line 2:** torch is needed to check which hardware (GPU or CPU) is available.

```
ROOT = Path(__file__).resolve().parent
DATA_DIR = ROOT / "data"
CKPT_DIR = ROOT / "checkpoints"
RESULTS_DIR = ROOT / "results"
```

- **Line 5:** __file__ is this script's location. .resolve() makes it a full path, and .parent takes the folder it sits in. So ROOT is the project folder.
- **Lines 6-8:** These build the paths of the three sub-folders. data/ holds the .npz datasets, checkpoints/ holds saved models and results/ holds training logs.

```
SAMPLE_RATE = 8000
DURATION = 5.0
N_SAMPLES = int(SAMPLE_RATE * DURATION)
```

- **Line 11:** Audio is converted to 8000 samples per second. Ship noise is low frequency, so a low rate keeps the data small.
- **Line 12:** Each clip is 5 seconds long.
- **Line 13:** 8000 × 5 = 40,000 samples per clip.

```
N_FFT = 1024
HOP_LENGTH = 313
N_MELS = 128
F_MIN = 20
F_MAX = 4000
```

- These are the spectrogram settings. train.py does not use them directly, but they decide the input shape.
- **N_FFT = 1024:** the window size for each Fourier transform, which sets the frequency detail.
- **HOP_LENGTH = 313:** how far the window moves each step. 40000 / 313 gives about 128 time frames.

- **N_MELS = 128:** the number of frequency bands, so the spectrogram is 128 × 128.
- **F_MIN and F_MAX:** only 20–4000 Hz is kept, which is where ship sound lives.

```
NUM_SAMPLES = 10000
SEED = 42
TRAIN_FRAC, VAL_FRAC, TEST_FRAC = 0.70, 0.15, 0.15
```

- **Line 23:** The number of synthetic clips to generate. Only the dataset builder uses it.
- **Line 24:** SEED makes random numbers repeatable, so you get the same results each run.
- **Line 25:** The 70/15/15 split into train, validation and test.

```
BATCH_SIZE = 64
EPOCHS = 20
LEARNING_RATE = 1e-3
NUM_WORKERS = 0
```

- **Line 28:** The model sees 64 clips at a time before it updates its weights.
- **Line 29:** At most 20 full passes over the training data.
- **Line 30:** The step size of weight updates (0.001).
- **Line 31:** Not used here, because data is loaded straight from memory.

```
def get_device():
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")
```

- It picks the fastest hardware automatically: an NVIDIA GPU first (cuda), then an Apple GPU (mps), otherwise the CPU. The model and data are later moved to whatever this returns.

```
def make_dirs():
    for d in (DATA_DIR, CKPT_DIR, RESULTS_DIR):
        d.mkdir(parents=True, exist_ok=True)
```

- It creates the three folders. parents=True also creates any missing parent folders, and exist_ok=True means no error if the folder already exists.

FILE 2: sonar_model.py (builds the neural network)

```
import torch
import torch.nn as nn
from torchvision import models
```

- **torch.nn** has the layers (convolution, linear and so on).
- **torchvision.models** has ready-made architectures like ResNet-18 and EfficientNet.

_build(arch, pretrained)

The leading underscore marks it as an internal helper.

```
if arch == "resnet18":
    w = models.ResNet18_Weights.DEFAULT if pretrained else None
    m = models.resnet18(weights=w)
```

- It chooses the weights: ImageNet-pretrained if pretrained=True, otherwise none (random). Starting from pretrained weights usually trains faster and better.
- It then creates the ResNet-18 network.

```
    old = m.conv1
    new = nn.Conv2d(1, old.out_channels, old.kernel_size, old.stride, old.padding,
bias=False)
```

- ResNet expects **3-channel colour images**, but a spectrogram has **1 channel**. So the first convolution layer is replaced with one that takes 1 input channel. Everything else (output channels, kernel size, stride, padding) is copied from the old layer.

```
    if pretrained:
        new.weight.data = old.weight.data.mean(dim=1, keepdim=True)
```

- To keep the pretrained knowledge, the old RGB weights are averaged into one channel. The new layer starts with useful filters instead of random ones.

```
    m.conv1 = new
    m.fc = nn.Linear(m.fc.in_features, 1)
```

- **m.conv1** = **new** swaps the new first layer into the network.
- **m.fc** is the final layer. The original outputs 1000 ImageNet classes. Here it is replaced with a layer that outputs **one number** (a logit). A positive value means ship and a negative value means no ship.

```
elif arch == "efficientnet_b0":
    ... (same idea)
    m.features[0][0] = new
    m.classifier = nn.Sequential(nn.Dropout(0.2), nn.Linear(m.classifier[1].in_features, 1))
```

- This does the same for EfficientNet-B0. Its first conv sits at features[0][0]. The new head adds **Dropout(0.2)**, which randomly switches off 20% of neurons during training to reduce overfitting, then a single-output linear layer.

```
else:
    raise ValueError(f"Unknown arch: {arch}")
return m
```

- An unknown name stops with a clear error, otherwise the finished model is returned.

build_model(arch="resnet18", pretrained=True)

```
try:
    return _build(arch, pretrained)
except Exception as e:
    if not pretrained:
        raise
    print("[warn] Could not load pretrained weights ...; training from scratch.")
    return _build(arch, False)
```

- It is a safety wrapper. If the pretrained weights cannot be downloaded (for example, no internet), it prints a warning and builds the model **without** pretrained weights instead of crashing. If it already was pretrained=False and still failed, the error is re-raised (raise).

count_params(m)

```
return sum(p.numel() for p in m.parameters())
```

- It adds up the number of weights in the model (numel() is the element count of each tensor). train.py prints this as “params: 11.2M”.

FILE 3: train.py (the training loop)

Imports (lines 1-9)

```
import os
os.environ.setdefault("PYTORCH_ENABLE_MPS_FALLBACK", "1")
```

- On Apple GPUs, if an operation is not supported, PyTorch falls back to the CPU instead of crashing. It **must be set before import torch**, which is why it comes first. setdefault does not overwrite it if you already set it.

```
import argparse, json, math, time
import numpy as np
import torch
import torch.nn as nn
import sonar_config as config
from sonar_model import build_model, count_params
```

- **argparse** reads command-line options such as --epochs.
- **json** saves the history file.
- **math** is used for ceil.
- **time** times each epoch.
- **numpy** loads the .npz data.
- **config** and **build_model/count_params** are the two files explained above.

load_split(name, device) (lines 12-16)

```
d = np.load(config.DATA_DIR / f"{name}.npz")
X = torch.from_numpy(d["X"]).to(device)
y = torch.from_numpy(d["y"]).float().to(device)
return X, y
```

- **Line 13:** Opens data/train.npz (or val.npz). The file holds X (spectrograms) and y (labels).
- **Line 14:** Converts the NumPy array to a PyTorch tensor and moves it to the GPU or CPU. The comment says it is float16 and kept on the device: half precision uses half the memory, and having all data on the GPU avoids slow transfers each batch.
- **Line 15:** The labels (0/1) become float, because the loss function needs floats.

augment(x) (lines 19-31)

Augmentation creates slightly changed copies of the data so the model does not memorise it.

```
B, _, H, W = x.shape
dev = x.device
```

- It unpacks the shape: batch size, channels (ignored), height 128 (frequency) and width 128 (time). dev is the device.

```
shifts = torch.randint(0, W, (B,), device=dev)
cols = (torch.arange(W, device=dev)[None, :] - shifts[:, None]) % W
x = x.gather(3, cols[:, None, None, :].expand(-1, 1, H, -1))
```

- **Time shift:** each clip gets a random shift amount.
- The second line builds shifted column indices. The % W wraps them around, so it is a **circular** shift.
- gather along the time axis (dim 3) applies it. Ship sounds should be detectable wherever they fall in time.

```
start = torch.randint(0, W, (B,), device=dev)
width = torch.randint(4, 17, (B,), device=dev)
t = torch.arange(W, device=dev)[None, :]
mask = (t >= start[:, None]) & (t < (start + width)[:, None]) & (torch.rand(B, device=dev) < 0.5)[:, None]
return x.masked_fill(mask[:, None, None, :], 0.0)
```

- **Time masking:** it picks a random start and a width of 4–16 frames.
- The mask is True for frames inside that range, and only for about 50% of the clips (torch.rand < 0.5).
- masked_fill(..., 0.0) blanks those frames. This forces the model not to depend on one short moment.

evaluate(model, X, y, loss_fn, bs=256) (lines 34-44)

```
@torch.no_grad()
```

- Turns off gradient tracking. It is not training, so this saves memory and time.

```
model.eval()
total_loss, correct = 0.0, 0
```

- **model.eval()** switches layers like Dropout and BatchNorm to inference behaviour. Then it sets up the counters.

```
for i in range(0, len(X), bs):
    xb = X[i:i + bs].unsqueeze(1).float()
    yb = y[i:i + bs]
    out = model(xb).squeeze(1)
```

- It goes through the validation data in chunks of 256.

- unsqueeze(1) adds the channel dimension, turning (B,128,128) into (B,1,128,128). .float() converts from float16.
- squeeze(1) turns the output (B,1) into (B,).

```
total_loss += loss_fn(out, yb).item() * len(yb)
correct += ((out > 0).float() == yb).sum().item()
```

- It adds the batch loss, weighted by batch size, so the final average is exact.
- out > 0 is the same as probability > 0.5. It counts how many predictions match the labels.

```
return total_loss / len(X), correct / len(X)
```

- It returns the average loss and the accuracy.

main() (lines 47-128)

Argument parsing (lines 48-58)

```
ap = argparse.ArgumentParser()
ap.add_argument("--arch", default="resnet18", choices=[...])
ap.add_argument("--epochs", type=int, default=config.EPOCHS)
...
args = ap.parse_args()
```

- It defines the options you can type on the command line, with defaults taken from sonar_config.py.
- **--patience** (default 5) is how many epochs without improvement before stopping.
- **--no-pretrained** is a flag that turns off pretrained weights.
- **--train-file/--val-file** choose which .npz to use.
- **--ckpt-dir** is where the model is saved.

Setup (lines 60-67)

```
config.make_dirs()
torch.manual_seed(config.SEED)
np.random.seed(config.SEED)
device = config.get_device()
use_amp = device.type == "cuda"
```

- It creates the folders and seeds the random generators (42) for repeatability.
- It selects the hardware. use_amp turns on **mixed precision** (faster 16-bit maths) only on NVIDIA GPUs.

```
Xtr, ytr = load_split(args.train_file, device)
Xva, yva = load_split(args.val_file, device)
```

- It loads training data and validation data. The validation data is never trained on, only used to check progress.

Model, loss, optimiser (lines 69-77)

```
model = build_model(args.arch, pretrained=not args.no_pretrained).to(device)
print(f"Arch: ... params: {count_params(model)/1e6:.1f}M ...")
```

- It builds the network, moves it to the device and prints a summary.

```
loss_fn = nn.BCEWithLogitsLoss()
```

- This is Binary Cross-Entropy on raw logits. It suits yes/no classification and includes the sigmoid inside, which is more numerically stable.

```
opt = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=1e-4)
```

- AdamW is the optimiser that updates the weights. weight_decay=1e-4 gently shrinks weights to reduce overfitting.

```
steps = math.ceil(len(Xtr) / args.batch)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=args.lr, epochs=args.epochs,
steps_per_epoch=steps)
```

- steps is the number of batches per epoch.
- OneCycleLR raises the learning rate to its maximum early on, then lowers it smoothly. This usually gives faster, more stable training.

```
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
```

- It scales the loss during 16-bit training so tiny gradients do not vanish. It does nothing if AMP is off.

Checkpoint setup (lines 79-82)

```
ckpt_dir = config.ROOT / args.ckpt_dir
ckpt_dir.mkdir(exist_ok=True)
ckpt_path = ckpt_dir / f"{args.arch}_best.pt"
history, best_val, bad = [], float("inf"), 0
```

- It builds the save path, e.g. checkpoints/resnet18_best.pt.
- history stores the per-epoch results. best_val starts at infinity so the first epoch always counts as an improvement. bad counts the epochs without improvement.

Epoch loop (lines 84-122)

```
print(f"\n{'epoch':>5} {'tr_loss':>8} ...")
for epoch in range(1, args.epochs + 1):
    t0 = time.time()
    model.train()
```

- It prints the table header and starts the epoch timer. model.train() switches on training behaviour (Dropout and BatchNorm updating).

```
perm = torch.randperm(len(Xtr)).to(device)
tl, tc, seen = 0.0, 0, 0
```

- It shuffles the order of the training samples each epoch, and resets the running loss, correct count and samples seen.

```
for i in range(0, len(Xtr), args.batch):
    idx = perm[i:i + args.batch]
    if len(idx) < 2:
        continue
```

- It takes the next 64 shuffled indices. A batch of 1 is skipped because **BatchNorm needs at least 2 samples** in training mode.

```
xb = augment(Xtr[idx].unsqueeze(1).float())
yb = ytr[idx]
```

- It picks the batch, adds the channel dimension, converts to float32 and applies the augmentation. The labels are picked with the same indices.

```
opt.zero_grad(set_to_none=True)
```

- It clears the gradients from the previous step. set_to_none=True is slightly faster.

```
with torch.autocast(device_type="cuda", enabled=use_amp):
    out = model(xb).squeeze(1)
    loss = loss_fn(out.float(), yb)
```

- **The forward pass.** The model predicts a logit per clip, and the loss compares it with the true label. out.float() computes the loss in full precision for stability.

```
scaler.scale(loss).backward()
scaler.step(opt)
scaler.update()
sched.step()
```

- **backward()** computes the gradients (backpropagation).

- **scaler.step(opt)** updates the weights.
- **scaler.update()** adjusts the scale factor.
- **sched.step()** advances the learning-rate schedule. It runs every batch, as OneCycleLR requires.

```
tl += loss.item() * len(idx)
tc += ((out > 0).float() == yb).sum().item()
seen += len(idx)
```

- It accumulates the training loss, the correct predictions and the number of samples, for the epoch averages.

```
va_loss, va_acc = evaluate(model, Xva, yva, loss_fn)
row = dict(epoch=epoch, train_loss=tl/seen, train_acc=tc/seen, val_loss=va_loss,
val_acc=va_acc)
history.append(row)
print(...)
```

- After all batches, it tests on the validation set, stores the metrics for this epoch and prints them.

```
if va_loss < best_val:
    best_val, bad = va_loss, 0
    torch.save({"arch": ..., "state_dict": model.state_dict(), "epoch": epoch,
                "val_loss": va_loss, "val_acc": va_acc}, ckpt_path)
else:
    bad += 1
    if bad >= args.patience:
        print("Early stopping ...")
        break
```

- **Save the best model.** If validation loss improved, it saves the weights and info. Later GUI loading uses ck["state_dict"].
- **Early stopping.** If there is no improvement for 5 epochs in a row, training stops, since further training would only overfit.

Finishing (lines 124-128)

```
with open(config.RESULTS_DIR / f"history_{args.arch}.json", "w") as f:
    json.dump(history, f, indent=2)
best = min(history, key=lambda r: r["val_loss"])
print(f"\nBest epoch: ...")
print(f"Checkpoint: {ckpt_path}")
```

- It writes the epoch-by-epoch history to results/history_resnet18.json, which is useful for plotting curves.

- It finds the epoch with the lowest validation loss and prints it with the checkpoint location.

```
if __name__ == "__main__":
    main()
```

- main() runs only when you execute python train.py directly, not when another file imports it. (train_mixed.py imports augment from this file, so this guard matters.)

If you want this as a Word or PDF document to keep, I can generate it.

ok now tell me the full flow what will happen if i clicked on load button and i selected the audio .wav file till to get output

Flow: clicking “Load audio” → selecting a .wav → final output

The first item in the model dropdown is “ResNet-18 v2 (real-data tested)” (if checkpoints_v2/ exists), so I assume it is selected. Where the other models behave differently, I say so.

Stage 1: Button click and file selection (main/GUI thread)

```
User clicks "Load audio"
 └─ btn_load.clicked  (wired in Main.__init__)
    └─ Main.pick_file()
       ├─ QFileDialog.getOpenFileName(...)   → opens the file picker (filters: *.wav *.flac *.mp3 *.ogg)
       ├─ user selects  "ship.wav"           → returns the full path
       ├─ if path is empty (user pressed Cancel) → nothing happens
       └─ Main.start({"kind": "file", "path": path})
```

Stage 2: Main.start(job) prepares the analysis (still the GUI thread)

```
Main.start(job)
 ├─ if self.worker is running → return        (ignores a double click while busy)
 ├─ arch, d = self.combo.currentData()        → e.g. ("resnet18", ".../checkpoints_v2")
 ├─ self.is_v2 = d.endswith("checkpoints_v2")   → True
```

```
├─ self.kind = "file"                    → used later to pick the real-file cutoff
├─ job.update(arch=..., ckpt_dir=..., v2=...)
├─ self.set_busy(True)                   → disables the 3 buttons and the dropdown
├─ self.set_visual("scanning")
│    ├─ radar.set_state("scanning", 0.0)     → SonarRadar: sweep speeds up to 230°/s,
random noise blips appear
│    ├─ banner.set_state("scanning")        → banner text: "ANALYSING - building
spectrogram and running the CNN"
│    └─ strip.set_ship(False)               → hydrophone strip shows noise only
├─ self.bar.setValue(0) ; _style_bar(th.PRIMARY)   → progress bar reset, blue
├─ self.t_start = time.time()              → for the minimum animation time later
├─ self.worker = Worker(job)                 → creates a QThread
├─ worker.done.connect(self.on_done)
├─ worker.failed.connect(self.on_failed)
└─ worker.start()                        → starts Worker.run() in the background thread
```

The analysis runs in a **separate thread** so the window and its animations do not freeze.

Stage 3: `Worker.run()` does the real work (background thread)

3a. Read the audio

```
j["kind"] == "file"  →  librosa.load(path, sr=8000, mono=True)
```

This does three things:

- Decodes the file.
- Resamples it to 8000 Hz, whatever the original rate was.
- Mixes it down to a single channel (mono).

It returns `wave`, a 1-D float array of samples between -1 and 1.

3b. Validate

```
if len(wave) < 8000:  raise ValueError("Audio is shorter than 1 second.")
```

This jumps to the `except` block, which emits `failed`. See the failure path at the end.

3c. Load the model

```
device = config.get_device()              → cuda / mps / cpu
model  = get_model(arch, ckpt_dir, device)

get_model(...)
 ├─ key = (arch, ckpt_dir)
```

```
├─ if key in _MODELS → return the cached model      (2nd+ analysis is faster)
└─ otherwise:
    ├─ torch.load(checkpoints_v2/resnet18_best.pt)     → dict with state_dict
    ├─ build_model("resnet18", pretrained=False)       → empty ResNet-18 (1-channel input,
1 output)
    ├─ .to(device)
    ├─ load_state_dict(ck["state_dict"])               → fills it with the trained weights
    ├─ m.eval()                                        → inference mode
    └─ stores it in _MODELS
```

3d. Cut into windows and predict each one

```
for t0, chunk in split_windows(wave):
```

infer.split_windows(wave):

- If the audio is shorter than 5 s, it pads with zeros to 5 s.
- It cuts the audio into non-overlapping 5-second windows (40,000 samples each).
- A leftover tail is kept only if it is at least 2.5 s, and it is zero-padded to 5 s. A shorter tail is dropped.
- It returns a list of (start_time_seconds, chunk).

Examples: a 12 s file gives 2 windows (0–5 and 5–10, with the 2 s tail dropped). A 13 s file gives 3 windows (the 3 s tail is padded).

For **each window**:

```
spec = sim.to_spectrogram(chunk)
 ├─ librosa.feature.melspectrogram(...)    → 128 mel bands × ~128 time frames
 ├─ librosa.power_to_db(..., top_db=80)    → converts to decibels
 └─ (S_db - mean) / std                    → per-clip standardisation
                                  → float32 array of shape (128, 128)


x = torch.from_numpy(spec)[None, None].to(device)    → shape (1, 1, 128, 128)
z = model(x).squeeze().item()                        → one raw score (logit)
prob = 1 / (1 + exp(-z))                             → sigmoid, a value from 0 to 1


starts.append(t0); logits.append(z); probs.append(prob); specs.append(spec)
```

All of it runs inside torch.no_grad(), so no training machinery is used.

3e. Send results back

```
self.done.emit(dict(starts=..., probs=..., specs=..., logits=...))
```

Qt safely delivers the signal to the main thread, which triggers on_done.

Stage 4: Showing results (back on the GUI thread)

4a. on_done(res)

```
wait = max(0, 1600 - elapsed_ms)
QTimer.singleShot(wait, lambda: self.show_result(res))
```

If the analysis took less than 1.6 s, it waits for the rest of that time so the scanning animation is visible. If it took longer, it shows the result immediately.

4b. show_result(res)

```
self.result = res
self.sel = argmax(res["probs"])       → selects the window with the highest score
self.set_busy(False)                  → buttons re-enabled
self.apply_results()
```

4c. apply_results() (decides ship or not, and updates the screen)

1. Scoring

```
if self.is_v2:                              # v2 model
   base = 9.23 (because kind == "file")       # cutoff calibrated on real-data holdout tests
   cut  = base + (slider - 50) / 5            # slider shifts the cutoff
   probs = sigmoid(z - cut)  for each window    # logits shifted by the cutoff
   thr = 0.5
else:                                       # other models
   thr = slider / 100
   probs = result["probs"]
```

For the v2 model with a real file, a window counts as ship only if its raw logit is above about 9.23. This is because the v2 model was trained on synthetic data and needs a higher cutoff on real recordings to avoid false alarms. With the default slider (50) the shift is 0. For the v2 model the column is labelled “Score”, otherwise “P(ship)”.

2. Table

```
table.setHorizontalHeaderLabels(["Window", "Score"/"P(ship)", "Verdict"])
table.blockSignals(True)
for each window i:
   row = ["0-5 s", "0.873", "SHIP"/"no ship"]
```

```
   colour verdict: red (th.ALERT) if flagged, green (th.OK) otherwise
table.blockSignals(False)
table.selectRow(self.sel)
```

Signals are blocked while filling so `on_select` does not fire for every row.

3. Overall verdict

```
peak = max(probs)
ship = peak > thr                  → the file is "ship" if ANY window is flagged
self.set_visual("ship" or "clear", peak)
   ├─ radar.set_state(...)     → "ship": a target blip appears at a random bearing; "clear":
calm sweep
   ├─ banner.set_state(...)    → "CONTACT DETECTED ..." (red) or "AREA CLEAR ..." (green)
   └─ strip.set_ship(ship)     → waveform adds harmonic tones if a ship was detected
```

4. Progress bar

```
bar.setValue(round(peak * 100))
bar.setFormat("Peak score 87%")
_style_bar(ALERT red or OK green)
```

5. Spectrogram

```
spec_view.set_spec(result["specs"][self.sel])
 ├─ idx = clip((spec + 2.5) / 6.5 * 255, 0, 255) → uint8      (maps values to 0–255)
 ├─ rgb = LUT[idx][::-1]                                      (magma colours, flipped so low frequencies
are at the bottom)
 ├─ QImage(...).copy()
 └─ self.update() → SpecView.paintEvent() draws the rounded card, title
"SPECTROGRAM", and the image
```

The spectrogram shown is that of the highest-scoring window.

What you see at the end

- **Banner:** red “CONTACT DETECTED” or green “AREA CLEAR”.
- **Radar:** a target blip if a ship was found.
- **Progress bar:** the peak score across all windows.
- **Table:** one row per 5 s window with score and verdict.
- **Spectrogram:** the most suspicious window.

After the result (without re-running the model)

- **Move the threshold slider:** on_threshold(v) → update_thr_label() → apply_results(). It re-scores from the stored logits, so it is instant.
- **Click a table row:** on_select() → spec_view.set_spec(specs[row]).
- **Click Load audio again:** the whole flow repeats, and the model is reused from cache.

Failure path

If the file is unreadable or shorter than 1 s, Worker.run emits failed(msg):

```
on_failed(msg) → set_busy(False) → set_visual("idle") → QMessageBox.critical("Analysis failed", msg)
```

One-line summary

```
click Load → pick_file → QFileDialog → start → set_busy/set_visual("scanning") → Worker.start
 → [thread] librosa.load → get_model → split_windows → (to_spectrogram → model → sigmoid) per window → done.emit
 → on_done → show_result → apply_results → table + banner + radar + bar + spectrogram
```